

THE SOFTWARE ENGINEER'S MENTAL MAP

A 30-Day Curriculum for Building Stack-Agnostic Understanding

Philosophy: Software engineering, stripped of any specific stack, is about three things: managing complexity, making tradeoffs, and communicating decisions. Every pattern, architecture, and concept you encounter is a response to one of these. That's your top-level map.

Structure: Four weeks, each a layer of abstraction. Week 1 zooms into code, Week 4 zooms out to engineering as a discipline. DSA runs as a parallel thread. Each day produces atomic Zettelkasten notes that build your permanent knowledge base.

Daily Format: 4 hours per day. Roughly 2 hours reading, 1 hour note-writing, 1 hour applying (exercises or DSA). Adjust the ratio based on how the material lands.

Recommended Resources

You don't need to buy all of these. Pick one primary resource per week and supplement with free articles.

Concept Area	Primary Resource	Free Alternative
Design Patterns & SOLID	Head First Design Patterns (2nd ed.)	Refactoring Guru (refactoring.guru)
Architecture	Fundamentals of Software Architecture (Richards & Ford)	Martin Fowler's blog (martinfowler.com)
System Design	System Design Interview Vol. 1 (Alex Xu)	ByteByteGo blog (blog.bytebytego.com)
Databases	Designing Data-Intensive Applications (Kleppmann)	Use the Index, Luke (use-the-index-luke.com)
DSA	Grokking Algorithms (Bhargava)	NeetCode roadmap (neetcode.io)

Testing	Unit Testing Principles (Khorikov)	Martin Fowler's testing articles
Technical Thinking	A Philosophy of Software Design (Ousterhout)	No Silver Bullet essay (free PDF)

WEEK 1 CODE-LEVEL THINKING

This week you formalize what you already do intuitively. Design patterns, SOLID principles, fundamental data structures. You'll move from 'I use MVC because Laravel uses it' to 'I understand why MVC exists, what problem it solves, and when it breaks down.'

Central Question: *How do we organize code so it can change without breaking?*

Day 1: Why Software Engineering Exists

Focus	Complexity as the fundamental problem. What makes software hard is not writing code — it is managing change over time.
Schedule	Hour 1: Read 'No Silver Bullet' by Fred Brooks (free online). Take notes on essential vs accidental complexity. Hour 2: Read about coupling and cohesion. Map these concepts onto a Laravel controller you've written — where is it tightly coupled? Why? Hour 3: Study separation of concerns (SoC). Understand it as the ONE principle that generates most patterns. Write atomic notes. Hour 4: Read about abstraction as a tool for managing complexity. Create your first Zettelkasten index note: 'Software Complexity.'
Atomic Notes	• Essential vs accidental complexity (#complexity #brooks) • Coupling: what it is, why it matters (#coupling #modularity) • Cohesion: the goal of good module design (#cohesion #modularity) • Separation of concerns as a meta-principle (#SoC #architecture) • Abstraction as complexity management (#abstraction #fundamentals)

Day 2: SOLID Principles — Part 1

Focus	Single Responsibility (SRP) and Open/Closed (OCP). These aren't rules — they're heuristics for managing change.
Schedule	Hour 1: Read about SRP. Key insight: 'a class should have one reason to change' means one stakeholder, not one method. Hour 2: Take a Laravel model you've written. Identify all the 'reasons to change' it has. How would SRP restructure it? Hour 3: Read about OCP. Connect it to polymorphism — this is how you extend behavior without modifying existing code. Hour 4: Write atomic notes. Key connection: SRP + OCP together explain why Laravel uses Service Providers.
Atomic Notes	• SRP: one reason to change, not one thing (#SOLID #SRP #modularity) • OCP: extend without modifying (#SOLID #OCP #polymorphism)

	• How SRP and OCP relate to each other (#SOLID #connections) • Laravel Service Providers as OCP in practice (#SOLID #laravel #example)
--	---

Day 3: SOLID Principles — Part 2

Focus	Liskov Substitution (LSP), Interface Segregation (ISP), and Dependency Inversion (DIP).
Schedule	Hour 1: Read about LSP. Core idea: subtypes must be substitutable. The classic Rectangle/Square violation. Why this matters for APIs. Hour 2: Read about ISP. Connect to cohesion — fat interfaces force unnecessary dependencies. Think about Laravel’s contracts. Hour 3: Read about DIP. This is the most architecturally important SOLID principle. High-level modules should not depend on low-level details. Hour 4: Map all 5 SOLID principles onto your mental model. Write a ‘hub note’ connecting them. Mini-exercise: refactor a controller to use DIP.
Atomic Notes	• LSP: substitutability as a contract (#SOLID #LSP #contracts) • ISP: small interfaces, focused dependencies (#SOLID #ISP #cohesion) • DIP: depend on abstractions, not concretions (#SOLID #DIP #architecture) • SOLID principles hub note: how all 5 connect (#SOLID #hub) • Laravel’s IoC container as DIP implementation (#SOLID #DIP #laravel)

Day 4: Design Patterns — Creational & Structural

Focus	Factory, Builder, Adapter, Decorator. Not memorizing patterns — understanding what problems they solve.
Schedule	Hour 1: Read about Factory pattern. When you need to decouple object creation from usage. Map to Laravel’s factory helpers. Hour 2: Read about Builder pattern. When construction is complex. Think about Laravel’s query builder — this IS the Builder pattern. Hour 3: Read about Adapter and Decorator. Adapter: making incompatible interfaces work together. Decorator: adding behavior without subclassing. Hour 4: Write atomic notes for each pattern using the template: Problem → Solution → Tradeoff → Example. Connect back to SOLID.
Atomic Notes	• Factory: decoupling creation from use (#patterns #creational #factory) • Builder: step-by-step complex object construction (#patterns #creational #builder) • Adapter: interface translation layer (#patterns #structural #adapter) • Decorator: composable behavior extension (#patterns #structural)

	#decorator) • Pattern template: Problem → Solution → Tradeoff (#patterns #meta)
--	---

Day 5: Design Patterns — Behavioral

Focus	Observer, Strategy, Command. These patterns show up constantly in both frontend and backend.
Schedule	Hour 1: Read about Observer pattern. This is React's state management, Laravel's events, pub/sub — all the same pattern. Hour 2: Read about Strategy pattern. Swappable algorithms. Connect to OCP — Strategy IS how you implement OCP. Hour 3: Read about Command pattern. Encapsulating actions as objects. Think about job queues, undo/redo, CQRS. Hour 4: Write notes. Create a 'Design Patterns Map' hub note that organizes all patterns by the problem they solve, not their category.
Atomic Notes	• Observer: decoupled event notification (#patterns #behavioral #observer #events) • Strategy: swappable algorithms at runtime (#patterns #behavioral #strategy #OCP) • Command: actions as first-class objects (#patterns #behavioral #command #CQRS) • Design Patterns Map: organized by problem, not category (#patterns #hub)

Day 6: Data Structures — Fundamentals

Focus	Arrays, Linked Lists, Stacks, Queues. Not just what they are — what operations they optimize for and why.
Schedule	Hour 1: Study Big-O notation. Not just memorize — understand WHY array access is O(1) (contiguous memory) and search is O(n). Hour 2: Arrays vs Linked Lists. Key insight: it's about memory layout and what operations you need most (access vs insert). Hour 3: Stacks and Queues as constrained data structures. LIFO vs FIFO. Where they appear: call stack, event loop, BFS/DFS. Hour 4: Write atomic notes. Key framework: every data structure is a tradeoff between read, write, and memory. Start a DSA hub note.
Atomic Notes	• Big-O: measuring algorithmic efficiency (#DSA #bigO #fundamentals) • Arrays: contiguous memory, O(1) access (#DSA #arrays #memory) • Linked Lists: pointer-based, O(1) insert (#DSA #linkedLists #memory)

	• Stacks and Queues: constrained access patterns (#DSA #stacks #queues) • Data structure tradeoff framework: read vs write vs memory (#DSA #hub #tradeoffs)
--	--

Day 7: Hash Maps & Trees + Week Review

Focus	Hash tables, binary trees, BSTs. Then consolidate Week 1 into your knowledge map.
Schedule	Hour 1: Study hash tables. Why $O(1)$ average lookup. Hash collisions. Why JavaScript objects and PHP arrays are hash maps underneath. Hour 2: Study binary trees and BSTs. Why $O(\log n)$ search. When trees beat arrays. Connection to database indexes (B-trees). Hour 3: Review all Week 1 notes. Look for connections you missed. Update hub notes. Create a 'Week 1 Map' index. Hour 4: Mini-exercise: given a real-world problem (e.g., implement a cache), reason about which data structure and which pattern to use. Write up your reasoning.
Atomic Notes	• Hash tables: key-value lookup via hashing (#DSA #hashMaps #hashing) • Binary trees: hierarchical $O(\log n)$ search (#DSA #trees #BST) • B-trees: why databases use tree variants (#DSA #trees #databases #indexes) • Week 1 Map: Code-Level Thinking index (#hub #week1)

WEEK 2 ARCHITECTURE-LEVEL THINKING

Zoom out from code to systems. How do components communicate? Where do you draw boundaries? How does data flow? You'll stop thinking 'Laravel app' and start thinking 'distributed system with a PHP backend.'

Central Question: *How do we structure systems so they can scale, evolve, and survive change?*

Day 8: Architectural Thinking

Focus	Moving from code-level to system-level. What is architecture? It's the decisions that are hard to change later.
Schedule	Hour 1: Read about what software architecture actually means. Key insight: architecture = the important stuff that's hard to reverse. Hour 2: Study Layered Architecture. Presentation → Business Logic → Data. Why Laravel is layered. What 'layers' actually enforce. Hour 3: Read about the difference between architecture and design. Architecture: system shape. Design: component internals. Both matter. Hour 4: Write notes. Key connection: SOLID principles operate at design level, architectural patterns operate at system level. Same goals, different scale.
Atomic Notes	• Architecture: decisions that are hard to reverse (#architecture #fundamentals) • Layered architecture: enforcing dependency direction (#architecture #layers) • Architecture vs Design: system shape vs component internals (#architecture #design #scale) • Scale principle: same concepts apply at different zoom levels (#meta #scale)

Day 9: Monoliths, Services, and the Spectrum

Focus	There's no monolith vs microservices binary. It's a spectrum of decomposition strategies.
Schedule	Hour 1: Study the modular monolith. Why most successful systems start here. How Laravel apps are usually structured as monoliths. Hour 2: Read about microservices. What problem they actually solve (team scaling, not technical). When they help vs when they hurt. Hour 3: Study the spectrum: monolith → modular monolith → SOA → microservices. Key insight: you're choosing where to put complexity. Hour 4: Write notes. DSA time: study graphs (nodes + edges) and graph

	traversal (BFS, DFS). Systems ARE graphs of services.
Atomic Notes	• Modular monolith: bounded modules in one deployable (#architecture #monolith #modular) • Microservices solve team scaling, not technical problems (#architecture #microservices #tradeoffs) • Architecture spectrum: where you put the complexity boundary (#architecture #spectrum #hub) • Graphs: nodes, edges, and traversal (#DSA #graphs #BFS #DFS)

Day 10: API Design Philosophy

Focus	REST, GraphQL, RPC — each is a different answer to ‘how should clients talk to servers?’
Schedule	Hour 1: Study REST deeply. It’s about resources and representations, not just HTTP verbs. HATEOAS. Why most ‘REST’ APIs aren’t truly RESTful. Hour 2: Study GraphQL. What problem it solves (over-fetching, under-fetching). When it adds unnecessary complexity. Hour 3: Study RPC (gRPC, tRPC). When direct function calls make more sense than resource-based thinking. Internal vs external APIs. Hour 4: Write notes. Key framework: API style should match your domain’s nature and your client’s needs. Create API Design hub note.
Atomic Notes	• REST: resource-oriented communication (#API #REST #resources) • GraphQL: client-driven data fetching (#API #GraphQL #flexibility) • RPC: action-oriented communication (#API #RPC #gRPC) • API style selection framework: domain shape × client needs (#API #hub #tradeoffs)

Day 11: Data Flow & State Management

Focus	How data moves through systems. This connects your React state knowledge to backend architecture.
Schedule	Hour 1: Study request/response cycle end-to-end. DNS → TCP → HTTP → Server → DB → Response. Understand every hop. Hour 2: Read about event-driven architecture. Events vs commands. Pub/sub. How this relates to Observer pattern at system scale. Hour 3: Study state management as a universal concept. Client state, server state, shared state. Why state is where bugs live. Hour 4: Write notes. DSA time: study sorting algorithms (quicksort, mergesort). Focus on divide-and-conquer as a problem-solving strategy.

Atomic Notes	• Request lifecycle: every hop from client to server and back (#architecture #HTTP #networking) • Event-driven architecture: Observer pattern at system scale (#architecture #events #patterns) • State as the source of bugs: managing mutable shared state (#state #complexity #bugs) • Divide and conquer: breaking problems into subproblems (#DSA #sorting #strategy)
---------------------	---

Day 12: Caching, Queues, and Async

Focus	The infrastructure patterns that make systems fast and resilient.
Schedule	Hour 1: Study caching strategies. Cache-aside, write-through, write-back. Cache invalidation as the hardest problem in CS. Hour 2: Read about message queues (Redis, RabbitMQ concepts). Why async processing exists. How Laravel's queue system works architecturally. Hour 3: Study the concepts of synchronous vs asynchronous processing. Blocking vs non-blocking. How Node.js event loop differs from PHP's model. Hour 4: Write notes. Connect caching to data structures (hash maps!). Connect queues to the Queue data structure. Patterns repeating at different scales.
Atomic Notes	• Caching strategies: trading freshness for speed (#caching #performance #tradeoffs) • Cache invalidation: the hardest problem (#caching #complexity) • Message queues: decoupling time from processing (#architecture #queues #async) • Sync vs async: blocking models vs event-driven models (#async #concurrency #models)

Day 13: Hexagonal Architecture & Clean Architecture

Focus	The most influential architectural ideas for modern applications. Domain at the center, infrastructure at the edges.
Schedule	Hour 1: Study Hexagonal Architecture (Ports and Adapters). Core idea: your domain logic doesn't know about HTTP, databases, or frameworks. Hour 2: Read about Clean Architecture. Dependency rule: dependencies point inward. Connect to DIP from SOLID — same principle, system scale. Hour 3: Compare MVC (what you use) with Hexagonal. MVC is a UI

	pattern; Hexagonal is an application architecture. They're different zoom levels. Hour 4: Write notes. DSA time: study recursion and dynamic programming basics. Trees use recursion; DP uses it with memoization.
Atomic Notes	• Hexagonal architecture: domain-centric with ports and adapters (#architecture #hexagonal #ports) • Clean architecture: dependency rule points inward (#architecture #clean #DIP) • MVC vs Hexagonal: UI pattern vs application architecture (#architecture #MVC #scale) • Recursion and dynamic programming: reusing subproblem solutions (#DSA #recursion #DP)

Day 14: Domain-Driven Design Concepts + Week Review

Focus	DDD vocabulary: bounded contexts, aggregates, entities, value objects. Then consolidate Week 2.
Schedule	Hour 1: Read DDD strategic concepts. Bounded contexts: where a model applies. Ubiquitous language: shared vocabulary with stakeholders. Hour 2: Read DDD tactical concepts. Entities vs Value Objects. Aggregates as consistency boundaries. Repositories as data access abstraction. Hour 3: Review Week 2 notes. Update hub notes. Connect DDD to Clean Architecture (they complement each other). Create Week 2 Map. Hour 4: Exercise: take a feature you've built (e.g., user registration). Sketch how it would look in Layered, Hexagonal, and DDD styles.
Atomic Notes	• Bounded contexts: where a model's meaning applies (#DDD #strategic #boundaries) • Ubiquitous language: shared vocabulary reduces translation bugs (#DDD #communication) • Entities vs Value Objects: identity vs equality (#DDD #tactical #modeling) • Aggregates: consistency boundaries in your domain (#DDD #tactical #aggregates) • Week 2 Map: Architecture-Level Thinking index (#hub #week2)

WEEK 3 OPERATIONS-LEVEL THINKING

The layer most self-taught developers skip. Databases beyond CRUD, testing philosophy, CI/CD, error handling, security. This is what makes the difference between ‘it works’ and ‘it works reliably in production.’

Central Question: *How do we make systems that are reliable, secure, and maintainable over time?*

Day 15: Database Fundamentals Beyond CRUD

Focus	Databases are not just storage — they’re the most critical architectural decision in most systems.
Schedule	Hour 1: Study relational model deeply. Tables, rows, columns, keys. Why normalization exists — it’s about eliminating update anomalies. Hour 2: Study normalization forms (1NF through 3NF is enough). Then study when to denormalize and why (read performance). Hour 3: Study indexing. How B-tree indexes work. Why index choice matters more than query optimization. Composite indexes. Hour 4: Write notes. Connect B-trees to your DSA tree knowledge. Connect normalization to SoC — same principle, data level.
Atomic Notes	• Relational model: tables as mathematical relations (#databases #relational #fundamentals) • Normalization: eliminating update anomalies (#databases #normalization #consistency) • Denormalization: trading consistency for read speed (#databases #denormalization #tradeoffs) • B-tree indexes: how databases make queries fast (#databases #indexing #B-trees #DSA)

Day 16: Transactions, ACID, and Consistency

Focus	How databases guarantee correctness. This is where reliability engineering starts.
Schedule	Hour 1: Study ACID properties. Atomicity, Consistency, Isolation, Durability. What each actually means mechanically. Hour 2: Study transaction isolation levels. Read uncommitted through serializable. Why deadlocks happen. Hour 3: Study CAP theorem. Why distributed systems can’t have everything. Connect to the monolith vs microservices tradeoff. Hour 4: Write notes. DSA time: study hash table collision resolution (chaining vs open addressing). Connect to database index hash joins.

Atomic Notes	• ACID: guarantees for reliable data operations (#databases #ACID #reliability) • Isolation levels: trading consistency for concurrency (#databases #isolation #tradeoffs) • CAP theorem: the fundamental distributed systems constraint (#distributed #CAP #tradeoffs) • Hash collision strategies: chaining vs open addressing (#DSA #hashMaps #collisions)
---------------------	--

Day 17: SQL vs NoSQL and Data Modeling

Focus	Choosing the right database is a design decision. Understanding when each model fits.
Schedule	Hour 1: Study NoSQL categories: document (MongoDB), key-value (Redis), columnar (Cassandra), graph (Neo4j). What each optimizes for. Hour 2: Read about data modeling strategies. Relational modeling vs document modeling. Embedding vs referencing in NoSQL. Hour 3: Study when to use SQL vs NoSQL. Key insight: it's about your access patterns, not your data shape. Hour 4: Write notes. Create a 'Database Decision Framework' hub note. Exercise: model a blog system in both relational and document style.
Atomic Notes	• NoSQL categories: each optimizes for different access patterns (#databases #NoSQL #categories) • Data modeling: access patterns should drive schema design (#databases #modeling #accessPatterns) • SQL vs NoSQL: relational integrity vs flexible schemas (#databases #tradeoffs #hub) • Embedding vs referencing: denormalization choices in documents (#databases #NoSQL #modeling)

Day 18: Testing Philosophy

Focus	Testing isn't about catching bugs — it's about enabling change with confidence.
Schedule	Hour 1: Study the testing pyramid. Unit → Integration → E2E. Why the ratio matters. What each level actually validates. Hour 2: Read about test design. Arrange-Act-Assert. What makes a good test. Why testing behavior (not implementation) matters. Hour 3: Study test doubles: mocks, stubs, fakes, spies. When to use each. Why over-mocking is a design smell.

	Hour 4: Write notes. Key connection: testability is a proxy for good design. If code is hard to test, the design has coupling problems.
Atomic Notes	• Testing pyramid: unit/integration/E2E ratio and purpose (#testing #pyramid #strategy) • Test behavior, not implementation (#testing #design #principles) • Test doubles: mocks, stubs, fakes, spies (#testing #doubles #isolation) • Testability as a design quality signal (#testing #design #coupling #insight)

Day 19: CI/CD and DevOps Concepts

Focus	Automated pipelines as an engineering practice. How code gets from your machine to production.
Schedule	Hour 1: Study CI/CD pipeline concepts. Build → Test → Deploy. Why automation reduces risk. Feature flags and canary deployments. Hour 2: Read about environments: dev, staging, production. Why environment parity matters. Configuration management. Hour 3: Study containerization concepts (Docker). What containers solve: ‘works on my machine’ eliminated. Images, layers, registries. Hour 4: Write notes. DSA time: study heaps and priority queues. Connection: CI/CD pipelines use priority queues for job scheduling.
Atomic Notes	• CI/CD: automated confidence from commit to production (#devops #CICD #automation) • Environment parity: reducing surprise in production (#devops #environments #reliability) • Containers: reproducible isolated environments (#devops #containers #Docker) • Heaps and priority queues: efficient min/max extraction (#DSA #heaps #priorityQueues)

Day 20: Error Handling and Observability

Focus	How systems fail gracefully and how you know when something is wrong.
Schedule	Hour 1: Study error handling as architecture. Exception hierarchies. Fail-fast vs fail-safe. Circuit breaker pattern. Hour 2: Read about observability: logs, metrics, traces. The three pillars. Why you need all three, not just logs. Hour 3: Study resilience patterns: retry with backoff, bulkhead, timeout, fallback. How Netflix’s Hystrix popularized these. Hour 4: Write notes. Create ‘Reliability Engineering’ hub note. Connection:

	Circuit Breaker uses the State Machine pattern.
Atomic Notes	• Error handling as design: fail-fast vs fail-safe (#reliability #errors #design) • Observability: logs, metrics, traces as three pillars (#reliability #observability) • Circuit breaker: preventing cascade failures (#reliability #patterns #circuitBreaker) • Resilience patterns: retry, bulkhead, timeout, fallback (#reliability #patterns #hub)

Day 21: Security Fundamentals + Week Review

Focus	Security as a first-class concern, not an afterthought. Then consolidate Week 3.
Schedule	Hour 1: Study OWASP Top 10 at a conceptual level. SQL injection, XSS, CSRF — what each exploits and how defenses work. Hour 2: Read about authentication vs authorization. OAuth2 concepts. JWTs: what they are, when they're appropriate, common misuses. Hour 3: Review Week 3 notes. Update hub notes. Notice: Week 3 is about 'quality attributes' — reliability, security, performance. Create Week 3 Map. Hour 4: Exercise: list all the quality attributes of a system you've built. Where are the gaps? What would you change knowing what you know now?
Atomic Notes	• OWASP Top 10: the most common vulnerability categories (#security #OWASP #vulnerabilities) • AuthN vs AuthZ: identity vs permissions (#security #auth #fundamentals) • JWTs: stateless tokens and their tradeoffs (#security #JWT #tokens) • Quality attributes: the -ilities that define system excellence (#architecture #quality #hub) • Week 3 Map: Operations-Level Thinking index (#hub #week3)

WEEK 4 META-LEVEL THINKING

This is where everything connects. Technical decision-making, system design interviews, reading unfamiliar codebases, communicating technical ideas. This week transforms you from ‘person who codes’ to ‘engineer who solves problems.’

Central Question: *How do engineers think, decide, communicate, and create value?*

Day 22: Technical Decision-Making

Focus	How experienced engineers evaluate tradeoffs. Every decision is buy vs build, simple vs flexible, now vs later.
Schedule	Hour 1: Read about decision frameworks. Reversible vs irreversible decisions. Two-way doors (iterate fast) vs one-way doors (deliberate carefully). Hour 2: Study technical debt as a deliberate concept. Not all debt is bad — it’s about knowing when you’re borrowing and the interest rate. Hour 3: Read about Architecture Decision Records (ADRs). Status, context, decision, consequences. Practice writing one for a past decision. Hour 4: Write notes. DSA time: study time/space complexity tradeoffs in real algorithms. Decision-making IS tradeoff analysis.
Atomic Notes	• Reversible vs irreversible decisions (#decisions #strategy #risk) • Technical debt: intentional borrowing vs accidental mess (#techDebt #decisions #tradeoffs) • ADRs: documenting the why behind decisions (#documentation #ADR #communication) • Time vs space tradeoffs in algorithms (#DSA #tradeoffs #complexity)

Day 23: System Design Thinking — Part 1

Focus	How to approach system design problems. The framework you’ll use in interviews and real work.
Schedule	Hour 1: Study the system design framework: Requirements → Estimation → High-Level Design → Deep Dive → Tradeoffs. Hour 2: Practice: design a URL shortener. Walk through each step. Focus on back-of-envelope estimation (DAU, QPS, storage). Hour 3: Study horizontal vs vertical scaling. Load balancing concepts. Database replication and sharding. Hour 4: Write notes. Create ‘System Design Framework’ hub note with the step-by-step template you’ll reuse.
Atomic Notes	• System design framework: requirements → estimation → design → deep

	dive (#systemDesign #framework #interviews) • Back-of-envelope estimation: DAU, QPS, storage math (#systemDesign #estimation #math) • Horizontal vs vertical scaling (#systemDesign #scaling #tradeoffs) • Database sharding: distributing data across machines (#systemDesign #databases #sharding)
--	---

Day 24: System Design Thinking — Part 2

Focus	More complex design exercises. Building on yesterday's framework with harder problems.
Schedule	Hour 1: Practice: design a notification system. Push vs pull. Fan-out on write vs fan-out on read. Priority queues. Hour 2: Study CDNs, reverse proxies, and edge computing concepts. How content gets close to users. Hour 3: Practice: design a rate limiter. Token bucket vs sliding window. Where to place it architecturally. Hour 4: Write notes. Connect rate limiter to your DSA knowledge (sliding window is a data structure technique too!).
Atomic Notes	• Fan-out strategies: write-time vs read-time distribution (#systemDesign #fanOut #notifications) • CDNs and edge: bringing content close to users (#systemDesign #CDN #performance) • Rate limiting algorithms: token bucket, sliding window (#systemDesign #rateLimiting #algorithms) • Sliding window: same technique in DSA and system design (#connections #DSA #systemDesign)

Day 25: Reading and Navigating Unfamiliar Codebases

Focus	A skill no one teaches but everyone needs. How to approach code you didn't write.
Schedule	Hour 1: Study strategies for reading codebases. Start with entry points, trace request flow, identify patterns. README → routes → controllers → models. Hour 2: Read about code archaeology. Git blame, commit history, PR descriptions. Understanding WHY code is the way it is. Hour 3: Practice: clone an open-source project in a stack you don't know. Apply your strategies. Can you trace a request? Identify the architecture? Hour 4: Write notes. Key insight: your pattern knowledge from Weeks 1-3

	is what makes unfamiliar code readable. Patterns are the shared vocabulary.
Atomic Notes	• Codebase reading strategy: entry points → flow → patterns (#skills #codeReading #strategy) • Code archaeology: using git history to understand intent (#skills #git #archaeology) • Patterns as shared vocabulary across codebases (#patterns #codeReading #connections)

Day 26: Technical Communication

Focus	Writing about technical decisions clearly. This is what separates senior engineers from everyone else.
Schedule	Hour 1: Study RFC (Request for Comments) format. How to propose technical changes. Problem → Proposal → Alternatives → Migration. Hour 2: Practice writing a mini-RFC for a technical decision (e.g., ‘migrate from REST to GraphQL’). Focus on tradeoffs section. Hour 3: Read about documentation as code. README-driven development. API documentation. Why good docs reduce cognitive load. Hour 4: Write notes. DSA review: revisit your weakest DSA area. Key insight: technical writing IS compression — reducing complexity for readers.
Atomic Notes	• RFC format: proposing technical changes formally (#communication #RFC #process) • Technical writing as complexity compression (#communication #writing #abstraction) • Documentation as cognitive load reduction (#documentation #DX #complexity)

Day 27: Estimation, Scoping, and Planning

Focus	How to break down work, estimate effort, and communicate timelines. An underrated engineering skill.
Schedule	Hour 1: Study estimation techniques. T-shirt sizing, story points, time-based. Why humans are bad at estimation and what helps. Hour 2: Read about work breakdown structures. How to decompose a feature into implementable tasks. Identifying unknowns early. Hour 3: Study incremental delivery. MVP thinking. How to ship value early while building toward the full solution. Hour 4: Write notes. Exercise: take a feature idea and practice the full

	cycle: scope → estimate → break down → sequence.
Atomic Notes	• Estimation: managing uncertainty, not predicting the future (#planning #estimation #uncertainty) • Work breakdown: decomposition as a problem-solving tool (#planning #decomposition #strategy) • Incremental delivery: shipping value before shipping everything (#planning #MVP #delivery)

Day 28: The Engineer's Value Proposition + Final Review

Focus	Synthesize everything. What does a software engineer actually do? What makes one valuable?
Schedule	Hour 1: Review your complete Zettelkasten. Count your notes. Review your hub notes. See the map you've built. Hour 2: Write your personal engineering manifesto: what you believe about software, what problems you solve, how you think. Hour 3: Create your 'Master Map': a single index note that connects all 4 weeks. This is your memory palace. Hour 4: Identify your top 5 knowledge gaps. Create a learning roadmap for the next 3 months. The map keeps growing.
Atomic Notes	• Master Map: the complete software engineering knowledge graph (#hub #master #index) • Engineering value: managing complexity + making tradeoffs + communicating decisions (#meta #value #career) • Learning roadmap: gaps identified and next steps (#meta #growth #roadmap)

BUFFER DAYS Catch-Up & Synthesis

These two days are intentionally unstructured. Use them for whatever serves you most.

Day 29: Buffer / Deep Dive Day

Focus	Catch up on anything that felt rushed, or deep dive into the topic that interested you most.
Schedule	Hours 1-2: Revisit any day that felt incomplete. Re-read, re-write notes, fill gaps. Hours 3-4: Deep dive into your most interesting topic. Follow the curiosity — this is where expertise begins.

Atomic Notes	• Revised or expanded notes from your chosen deep-dive topic • New connections discovered during review
---------------------	--

Day 30: System Design Mock + Reflection

Focus	Put it all together. Walk through a complete system design problem using everything you've learned.
Schedule	Hours 1-2: Pick a system design problem (e.g., design Twitter, design a payment system). Walk through it end-to-end using your framework. Hours 3-4: Write a reflection: what changed in how you think about software? What would you tell yourself 30 days ago?
Atomic Notes	• System design walkthrough: complete problem-solving record (#systemDesign #practice) • 30-day reflection: growth documented (#meta #reflection #growth)

Zettelkasten Quick Guide

Atomic Note Template

Field	Description
Title	One concept per note. 'SRP: one reason to change' not 'SOLID principles.'
Content	Explain the concept in your own words. 3-8 sentences. If you need more, split into multiple notes.
Tags	Use the tags suggested in each day's plan. Add your own as connections emerge.
Connections	Link to related notes. 'This connects to [[DIP]] because...' The connections ARE the knowledge.
Source	Where you learned this. Book, article, or 'derived from experience.'
Example	A concrete example from your React/Laravel work. This anchors abstract concepts.

Hub Note Template

Hub notes are index notes that connect clusters of atomic notes. You'll create roughly one per major topic area. They're your map's landmarks.

Structure: Title (e.g., 'Design Patterns Map') → Overview paragraph → Linked atomic notes grouped by subtopic → Open questions you want to explore later.

Expected Outcomes

Metric	Target
Atomic notes created	~120-140 notes
Hub/index notes	~12-15 maps
DSA concepts covered	Arrays, linked lists, stacks, queues, hash maps, trees, graphs, heaps, sorting, recursion, DP basics
System design problems practiced	3-4 complete walkthroughs
Design patterns understood	10-12 key patterns with Problem → Solution → Tradeoff

	format
Interview readiness	Can discuss architecture, tradeoffs, and system design at junior-mid level

The map is not the territory — but a good map makes the territory navigable.